NanoAgent Plugins: Fundamentals and How to Build One

NanoAgent 0.5.0

1 What a plugin is

An agent saying it has finished a task does not show that its result is right. Judging an agent takes a precise task, an environment anyone can reproduce, and a verifier that checks the actual result. A NanoAgent plugin packages all three: typed tools served over MCP, tasks built from them, a deterministic verifier, and a front end. Every plugin's page is the same NanoAgent chat, whose agent runs the plugin's tasks and shows each result with its verdict. NanoAgent loads a plugin only when every rule of the `nanoagent.plugin/v1` contract holds, checks the rules again on every tool call, and lists every broken rule at once.

2 The fundamentals

1. **Typed in, typed out.** Every tool and task has an input class and an output class. They are the only shapes that cross the plugin's boundary, and every value is built into its class on the way in and out.
2. **Decisions are typed questions.** The model chooses: the next tool, and for each field an allowed value or an existing value of the same type. It does not invent values. Any model that answers typed questions can be used, and the contract does not depend on which.
3. **Rules in code, judgment in the model.** If a value follows from rules, a tool computes it. The model decides only what needs judgment.
4. **Every result is verified by code. LLM as judge is banned.** The verifier is plain, sandboxed, deterministic Python that decides every case. A task whose correctness code cannot check does not belong in a plugin.
5. **Natural language only at the end,** to explain the typed result to the person.

How questions are batched, what background the model sees and whether it is cached are harness settings. They change without touching any contract.

3 Anatomy

A plugin folder holds exactly these files, and nothing else at its top level. The Hello World starter has the smallest working example of each, and the arXiv Source plugin a full one, with network access, five tasks and a verifier module per tool.

File	What it holds
`contract.py`	Declarations only: `Tool` and `Task` Enums, slimconfig `Output` classes, and the `TOOLS` and `TASKS` tables pairing each with its input and output classes. Field descriptions become the JSON Schema everyone sees.
`nanoagent.yaml`	The manifest: id, version, the `nanoagent` version range, the MCP runtime, permissions (hosts it may reach, where it may write), and the tools and tasks tables. Each task has a `runner`, `tools`, `reference` and `instructions`.
`src/`	The tools and the MCP server. A tool takes its input class, returns its output class, and raises on failure. It writes only to its workspace.
`verifier/`	`verify_input` and `verify_output` run on every tool call, and `verify_task` runs once on the task's output. Each returns `valid`, `score` (0 to 1), `message` and `details`. Standard library only, sandboxed, and run twice: the verdicts must match.
`tests/`	The plugin's tests, including `test_verifier.py`.
`ui/`	`renderers.js` and `renderers.css`. Outputs are drawn from their JSON Schema unless a renderer draws their class.
`product/`	`product.yaml` (tagline, summary, highlights, steps) and `icon.svg`, the catalog page.
`run.sh`, `.env`	`run.sh` exits 0 when this machine can run the plugin. `.env` lists variable names, never values.
`README.md`, `LICENSE`, `pyproject.toml`, `uv.lock`, `.gitignore`	The template's standard files. The README has the sections Tools, Layout, and Develop and run.

4 How a task runs

A task's `runner` says who acts:

- `tool`: no model. One tool is called with the task's input, and its output is the task's, as in the starter's `say-hello`.
- `agent`: an LLM loop with the task's tools, ending with `submit`, which takes the task's output class, as in arXiv Source's `find-and-save-paper`.
- `decide`: the model only chooses: the next tool or `submit`, an Enum or `bool` member, or an existing value of exactly a field's type from the task's input or an earlier tool's output. NanoAgent calls the tools and builds every value. Every answer is checked against its options, and a task with a field that cannot be filled this way is refused when the plugin loads.

A task's `reference`, when set, is a tool outside the task's tools. NanoAgent runs it first, out of the model's sight, and pins its output as the ground truth for `verify_task`, as arXiv Source's `write-daily-digest` pins `list_daily_papers` and `its verifier` checks the digest against it. Otherwise it is `null`.

5 Building your own plugin

Install Git, Python 3.12 or later and `uv`, then start from the starter, replacing `my-task` with your plugin's name:

```
git clone https://github.com/zeyuyang8/mochi-hello-world.git my-task && cd my-task
```

1. **Rename it** in `nanoagent.yaml`, the README's title, `LICENSE` and `pyproject.toml` together.
2. **Declare the types** in `contract.py`. Use Enums and `bool`s for anything that is a choice.
3. **Write the tools** in `src/plugin/`, one module per tool, and register each in `__main__.py`. Put every rule-based computation in a tool.
4. **Describe the tools and tasks** in `nanoagent.yaml`, and grant only the permissions the plugin needs.
5. **Write the verifier** and its tests: deterministic code that decides every case, never a model. Test correct, incorrect, missing, malformed and boundary results, not only the correct one.
6. **Finish the front end**: renderers only where the default drawing falls short, plus `product.yaml` and `icon.svg`.
7. **Test it** with `uv sync -extra dev` and `uv run pytest`, and keep `uv.lock` current. Keep reports and run outputs outside the plugin folder.

The `starter's README` says how to add a tool or a task, and arXiv Source shows each piece at full size. The manifest's `nanoagent` field, such as `~=0.5.0`, is the range of NanoAgent versions the plugin runs on.

6 Publishing on Mochi Lab

1. Push the complete, tested plugin to a public GitHub repository.
2. Sign in at `https://mochi-lab.org`, and add an OpenRouter API key and choose the model under **Settings** → **AI**.
3. In **Marketplace**, select **+ Publish a plugin**, paste the repository URL (for a subfolder, `https://github.com/<owner>/<repo>/tree/<branch>/<folder>`), and select **Publish**. Fix any reported errors and publish again.
4. Once a workspace owner adds it, open the plugin, ask the chat for the task, and check the verdict on its result card.

Passing the publication checks shows the plugin follows the format, not that its task or verifier is correct. The verifier's tests and the runs' verdicts show that.